TECHNICAL CASE STUDY

Supply Chain Management Platform

Enterprise B2B SaaS — Microservices Architecture Case Study

Type: Full-Stack Distributed System | **Date:** April 2026

Backend: .NET 10 (C# 13) · 6 Microservices · Ocelot API Gateway

Frontend: Angular 21 · Standalone Components · Signal-Based State

Infrastructure: SQL Server · RabbitMQ · Redis · Docker Compose

Classification: Portfolio Case Study — Interview Ready

Table of Contents

1. Executive Summary
2. Business Problem & Context
3. System Architecture Overview
4. Technology Stack & Justification
5. Microservice Deep Dive — The 6 Domain Services
6. Design Patterns & Architectural Decisions
7. API Gateway — Cross-Cutting Concerns
8. Frontend Architecture — Angular 21 SPA
9. Security Architecture
10. Data Flow — Order Lifecycle (End-to-End Walkthrough)
11. Event-Driven Architecture & Messaging
12. Performance, Scalability & Resilience
13. Project Metrics & Complexity
14. Key Challenges & Solutions
15. Lessons Learned & Interview Talking Points

1. Executive Summary

The **Supply Chain Management Platform** is an enterprise-grade B2B SaaS application that digitizes the full dealer-to-delivery lifecycle for a manufacturing/distribution business. It handles **dealer onboarding, product catalog management, order processing with credit-hold workflows, logistics tracking with agent assignment, GST-compliant invoicing, and multi-channel notifications** — all built on a production-ready microservices architecture.

6	69+	80+	5
DOMAIN MICROSERVICES	ANGULAR COMPONENTS	REST API ENDPOINTS	USER ROLES (RBAC)

> **What makes this project stand out:** This isn't a CRUD application with a monolithic backend. It implements real distributed systems patterns — Saga orchestration, Transactional Outbox, Event-Driven Architecture, API Gateway with circuit breakers, and RBAC with JWT token revocation — all connected through a premium Angular SPA with role-adaptive dashboards.

2. Business Problem & Context

2.1 The Problem

A B2B manufacturing company manages hundreds of dealers who place bulk orders for products. The existing process is manual — orders via phone/email, credit checks on spreadsheets, shipment tracking through WhatsApp, and invoices generated in Tally. This leads to:

- **Order errors** — manual data entry causes wrong quantities, pricing mismatches, and duplicate orders
- **Credit risk** — no real-time visibility into dealer outstanding balances, leading to credit overexposure
- **Delivery blind spots** — no centralized tracking; dealers call warehouse staff for status updates
- **Invoice delays** — GST-compliant invoices generated days after delivery, causing payment collection lags
- **No analytics** — management lacks dashboards for revenue trends, top dealers, or inventory health

2.2 The Solution

A fully digital platform where:

- **Dealers** self-register, browse the catalog, place orders, and track shipments through a web portal
- **Admins** manage dealer approvals, product catalog, credit limits, and monitor analytics dashboards
- **Warehouse** staff process orders and manage inventory with automated stock alerts
- **Logistics** managers create shipments, assign delivery agents, and track last-mile delivery
- **Agents** update shipment status in real-time during delivery

2.3 User Roles & Access Matrix

ROLE	RESPONSIBILITIES	KEY CAPABILITIES
Admin	Platform governance & full operational visibility	All services — full CRUD, dealer approval/rejection, credit management, analytics
Dealer	Browse, order, pay, track	Catalog browse, cart/checkout, own orders, own invoices, shipment tracking
Warehouse	Stock management & fulfillment	Inventory management, order queue processing, mark ReadyForDispatch
Logistics	Shipment orchestration	Create shipments, assign agents, manage delivery lifecycle
Agent	Last-mile delivery execution	View assigned shipments, update delivery status, record delivery attempts

3. System Architecture Overview

The platform follows a **domain-driven microservices architecture** with an API gateway pattern, event-driven integration, and database-per-service isolation.

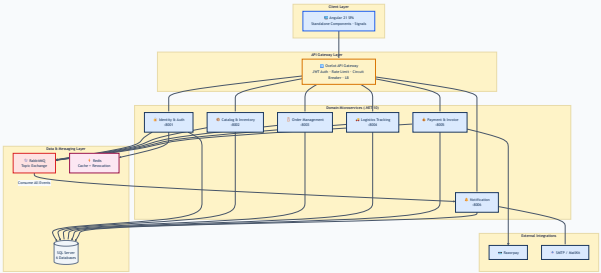


FIGURE 1 — HIGH-LEVEL SYSTEM ARCHITECTURE

Key Architecture Decisions:

- **Database-per-Service** — Each microservice owns its dedicated SQL Server database. No shared schemas.
- **Transactional Outbox + RabbitMQ** — Events are written to an outbox table atomically with business data, then published asynchronously.
- **Saga Orchestration** — The Order service coordinates multi-step workflows (stock reservation → credit check → approval) via an explicit state machine.
- **Single Entry Point** — All frontend traffic flows through the Ocelot API Gateway, which handles JWT validation, rate limiting, and routing.

4. Technology Stack & Justification

BACKEND RUNTIME

.NET 10 (C# 13)

Minimal APIs + Clean Architecture per service. Chosen for strong typing, high performance, and mature ecosystem for enterprise backends.

FRONTEND FRAMEWORK

Angular 21

Standalone components, Signal-based state management, lazy-loaded routes. Chosen for its opinionated structure suited to enterprise SPAs.

API GATEWAY

Ocelot + Polly

Route mapping, JWT pass-through, rate limiting (500/min per route), RoundRobin load balancing, circuit breaker with QoS policies.

PRIMARY DATABASE

SQL Server 2022

6 isolated databases (one per service). EF Core 10 Code-First with migrations. Transactional integrity within each bounded context.

MESSAGE BROKER

RabbitMQ 4.1

Topic exchange (`supplychain.events`) for async cross-service communication. Dead-letter queues for failed messages. Outbox dispatchers per service.

CACHE & IDEMPOTENCY

Redis 7.4

JWT token revocation store, idempotency key enforcement, response caching for high-read endpoints.

APPLICATION PIPELINE

MediatR + FluentValidation

CQRS-lite pattern with cross-cutting pipeline behaviors: validation, logging, idempotency, transaction management.

PAYMENT GATEWAY

Razorpay

Order creation, webhook signature verification, optional payment capture for prepaid orders.

PDF GENERATION

QuestPDF

GST-compliant invoice PDF generation with branded templates, IGST vs CGST+SGST breakdown.

EMAIL TRANSPORT

MailKit (SMTP)

Branded HTML email templates. Development emails routed to Mailpit for safe testing.

OBSERVABILITY

Serilog + Health Checks

Structured JSON logging, X-Correlation-Id tracing across all services, per-service health endpoints.

CONTAINERIZATION

Docker Compose

Redis, RabbitMQ, and Mailpit orchestrated via docker-compose.yml for consistent dev environments.

5. Microservice Deep Dive — The 6 Domain Services

5.1 Identity & Auth Service Port 8001

ASPECT	DETAILS
Bounded Context	User lifecycle, authentication, dealer onboarding, role management
Core Entities	Users, DealerProfiles, RefreshTokens, OtpRecords
Key Features	JWT access + httpOnly refresh token, OTP-based password reset, admin dealer approval/rejection workflow, credit limit sync to Payment service
Integration	→ Payment Service (HTTP PUT: sync credit limit on dealer approval) ← Notification Service (HTTP GET: resolve user email)
Events Published	DealerRegistered, DealerApproved, DealerRejected, PasswordResetRequested, PasswordResetCompleted

5.2 Catalog & Inventory Service Port 8002

ASPECT	DETAILS
Bounded Context	Product catalog, category hierarchy, stock management
Core Entities	Products, Categories, StockTransactions, StockSubscriptions
Key Features	Product CRUD with image management, category tree, **two-phase stock management** (SoftLock → HardDeduct), low-stock alerts, product reviews with moderation
Stock Model	`SoftLock` : reserve on order placement · `HardDeduct` : deduct on dispatch · `Release` : undo on cancellation · `Restock` : add back on return
Events Published	ProductCreated, StockUpdated, LowStockAlert

5.3 Order Management Service Port 8003

ASPECT	DETAILS
Bounded Context	Order lifecycle, credit-hold decisioning, returns, analytics
Core Entities	Orders (Aggregate Root), OrderLines, OrderStatusHistory, ReturnRequests, OrderSagaStates
State Machine	Placed → OnHold | Processing → ReadyForDispatch → InTransit → Delivered → Closed With branches: Exception, ReturnRequested → ReturnApproved/Rejected, Cancelled
Saga Coordinator	Orchestrates: Started → CreditCheckInProgress → AwaitingManualApproval | CompletedApproved | CompletedRejected | CompletedCancelled
Key Features	Credit check integration, bulk status updates, role-scoped transitions, purchase analytics (top dealers, products, daily revenue), idempotent order placement
Events Published	OrderPlaced, AdminApprovalRequired, OrderApproved, OrderCancelled, ReturnRequested, ReturnApproved, ReturnRejected

5.4 Logistics & Tracking Service Port 8004

ASPECT	DETAILS
Bounded Context	Shipment lifecycle, agent assignment, delivery tracking
Core Entities	Shipments, ShipmentEvents (timeline), Vehicles, DeliveryAttempts, ShipmentOpsStates
Shipment States	Created → Assigned → PickedUp → InTransit → OutForDelivery → Delivered | DeliveryFailed → Returned
Key Features	Agent assignment with accept/reject, delivery attempt tracking, ETA calculation, vehicle management, operations queue for exception handling
Events Published	ShipmentCreated, ShipmentAssigned, ShipmentStatusUpdated, ShipmentDelivered

5.5 Payment & Invoice Service Port 8005

ASPECT	DETAILS
Bounded Context	Credit management, invoice generation, payment processing
Core Entities	DealerCreditAccounts, Invoices, InvoiceLines, PaymentRecords, InvoiceWorkflowStates
Key Features	Dealer credit limit management, credit check API (consumed by Order service), GST-compliant invoice generation (IGST vs CGST+SGST), Razorpay integration, payment capture, invoice workflow state machine, PDF download
Events Published	InvoiceGenerated, PaymentCaptured, PaymentFailed, DealerCreditLimitUpdated

5.6 Notification Service `Port 8006`

ASPECT	DETAILS
Bounded Context	Notification delivery, preferences, email dispatch
Core Entities	Notifications, NotificationPreferences
Key Features	Consumes ALL domain events from RabbitMQ, routes to In-App or Email channel, HTML email templates via MailKit, digest batching, mark-as-read, user preference management
Channels	InApp (0) · Email (1) · SMS (2 — placeholder) · Push (3 — placeholder)

6. Design Patterns & Architectural Decisions

6.1 Saga Pattern — Order Orchestration

The Order service implements a **Saga Orchestration** pattern to coordinate the multi-step order creation workflow across Catalog (stock) and Payment (credit) services.

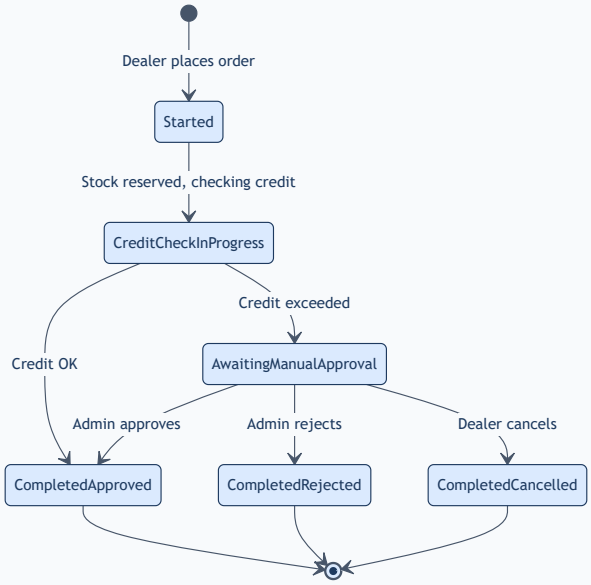


FIGURE 2 — ORDER SAGA STATE MACHINE

> **Why Saga instead of 2PC?** With 6 independent databases, a Two-Phase Commit is not feasible — it would require all databases to support XA transactions and would create blocking locks across services. The Saga pattern provides eventual consistency through compensating transactions (e.g., releasing stock locks on cancellation, restocking on return approval).

6.2 Transactional Outbox Pattern

Each service writes domain events to an `OutboxMessages` table in the same database transaction as the business data. A background `OutboxDispatcher` polls this table and publishes events to RabbitMQ.

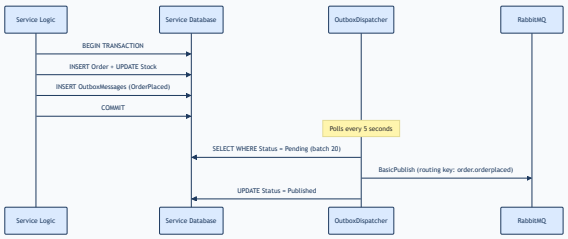


FIGURE 3 — TRANSACTIONAL OUTBOX PATTERN — GUARANTEED EVENT DELIVERY

> **Guarantee:** The outbox pattern ensures that if the business transaction succeeds, the event will eventually be published — even if RabbitMQ is temporarily down. The dispatcher retries up to 5 times before moving a message to Failed status.

6.3 Other Key Patterns

PATTERN	WHERE USED	PURPOSE
CQRS-Lite	All services via MediatR	Separate command/query pipelines with cross-cutting behaviors (validation, logging, transactions)
Aggregate Root (DDD)	OrderAggregate, Shipment	Encapsulate domain logic and enforce invariants — status transitions, stock rules
Repository Abstraction	IOrderRepository, IPaymentRepository	Decouple domain logic from EF Core persistence, enable unit testing with mocks
Gateway Aggregation	Ocelot routes to 6 services	Single entry point for frontend — simplifies CORS, auth, rate limiting
Idempotency Keys	Order creation via Redis	Prevent duplicate order placement from network retries or double-clicks
Circuit Breaker	Ocelot + Polly	3 failures → 10s break → prevents cascade failures from downstream outages

7. API Gateway — Cross-Cutting Concerns

The Ocelot API Gateway acts as the single entry point for all client requests to the backend microservices.

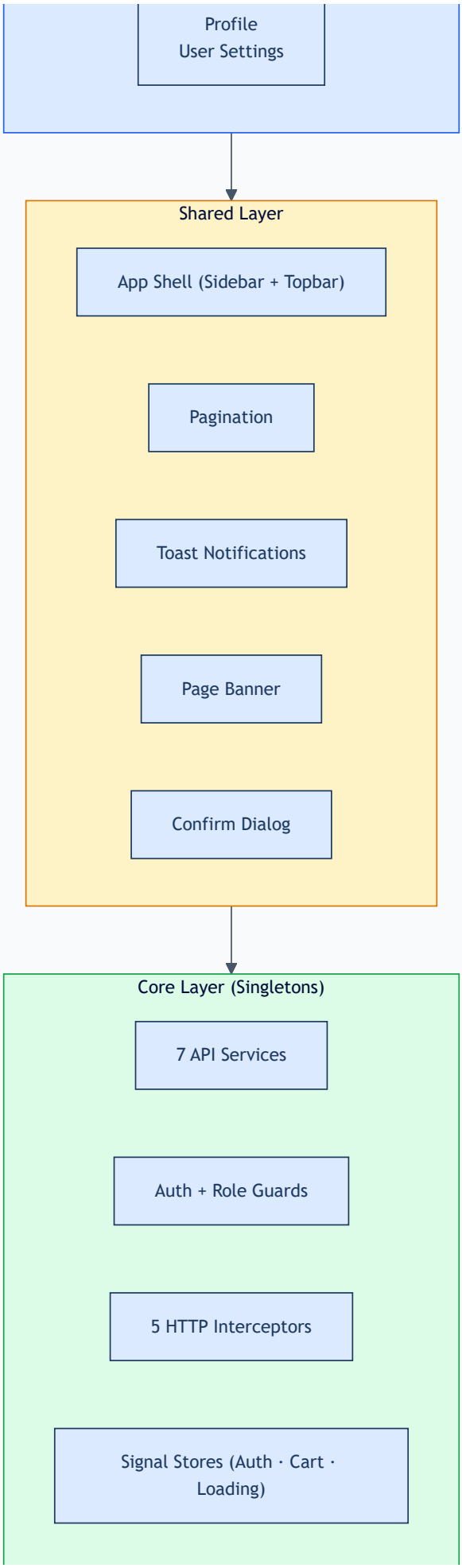
CONCERN	IMPLEMENTATION	DETAILS
Routing	URL prefix → service mapping	`/identity/*` → :8001, `/catalog/*` → :8002, `/orders/*` → :8003, etc.
Authentication	JWT Bearer validation	Validates issuer, audience, signature, and lifetime at the gateway level before forwarding
Rate Limiting	Per-route + Global	500 requests/min per route, 120 requests/min global per IP (keyed by `Oc-Client` header)
Load Balancing	RoundRobin via Ocelot	Demo routes (`/identity-lb/*`) distribute across primary (:8001) and secondary (:8101) instances
Circuit Breaker	Polly QoS policies	3 exceptions before breaking, 10s break duration, 5s request timeout
CORS	Configured on gateway only	`WithOrigins("http://localhost:4200")` — AllowCredentials for httpOnly cookies
Correlation Tracking	X-Correlation-Id header	Frontend generates UUID per request → gateway propagates → all services log with same correlation ID
Response Caching	File cache per route	15s TTL on identity GET routes to reduce redundant profile lookups

8. Frontend Architecture — Angular 21 SPA

The frontend is a premium B2B SaaS application built with Angular 21 using modern patterns — standalone components, signal-based state management, and lazy-loaded routes.

8.1 Architecture Layers

Presentation Layer (10 Feature Modules)
Auth
Login · Register · Forgot Password
Dashboard
Role-Adaptive Analytics
Catalog
Product List · Detail · Form
Cart & Checkout
Dealer Purchase Flow
Orders
List · Detail · Tracking
Logistics
Shipment List · Detail
Payments
Invoice List · Detail
Notifications
Center + Preferences
Admin
Dealer & Agent Mgmt


Profile
User Settings
Shared Layer
App Shell (Sidebar + Topbar)
Pagination
Toast Notifications
Page Banner
Confirm Dialog
Core Layer (Singletons)
7 API Services
Auth + Role Guards
5 HTTP Interceptors
Signal Stores (Auth · Cart ·
Loading)

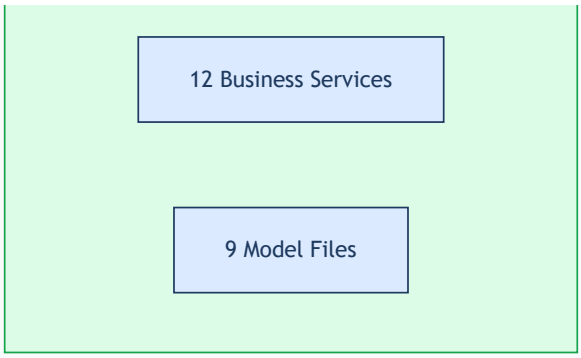


FIGURE 4 — FRONTEND THREE-LAYER ARCHITECTURE

8.2 Key Frontend Patterns

PATTERN	IMPLEMENTATION
Signal-Based State	AuthStore, CartStore, LoadingStore use Angular `signal()` and `computed()` — no NgRx dependency
Lazy Loading	All 24 feature components are lazy-loaded via `loadComponent()` in routes — minimal initial bundle
Functional Guards	`authGuard` (checks JWT + silent refresh), `roleGuard` (checks `route.data.roles[]`)
Functional Interceptors	5 interceptors chained: Correlation ID → Auth Bearer → UTC Date Normalization → Loading Spinner → Error Handler
Silent Token Refresh	On 401 error: refresh via httpOnly cookie → queue pending requests → retry with new token. Uses `BehaviorSubject` for concurrency.
Role-Adaptive UI	Dashboard, sidebar, and action buttons dynamically adjust based on `AuthStore.role()`. Admin sees all; Dealer sees own data.
Dev Proxy	`proxy.conf.json` maps 6 URL prefixes to gateway (:5000) — eliminates CORS in development

8.3 HTTP Interceptor Chain

FIGURE 5 — ANGULAR HTTP INTERCEPTOR PIPELINE (ORDER MATTERS)

9. Security Architecture

LAYER	MECHANISM	DETAILS
Authentication	JWT Bearer + httpOnly Refresh Cookie	Short-lived access token (15 min) + long-lived refresh token in httpOnly cookie. Silent refresh via interceptor.
Authorization	RBAC at 3 levels	① Gateway route-level auth ② Backend `[Authorize(Roles)]` per endpoint ③ Frontend `roleGuard` per route
Token Revocation	Redis JTI store	On logout, token's JTI (JWT ID) is added to Redis revocation set. All services check this on every request.
Internal API Security	X-Internal-Api-Key header	Service-to-service calls (e.g., Notification → Identity) protected by shared API key — not exposed to frontend.
Password Storage	Bcrypt hashing	No cleartext passwords stored. OTP-based reset flow with time-limited codes.
CORS Policy	Gateway-only, single origin	Only `http://localhost:4200` allowed. `AllowCredentials` for cookie-based refresh.
Rate Limiting	Gateway-level throttling	Prevents brute force on auth endpoints. 429 with structured JSON error response.

10. Data Flow — Order Lifecycle (End-to-End)

This traces a complete order from dealer placement through delivery, showing every service touched.

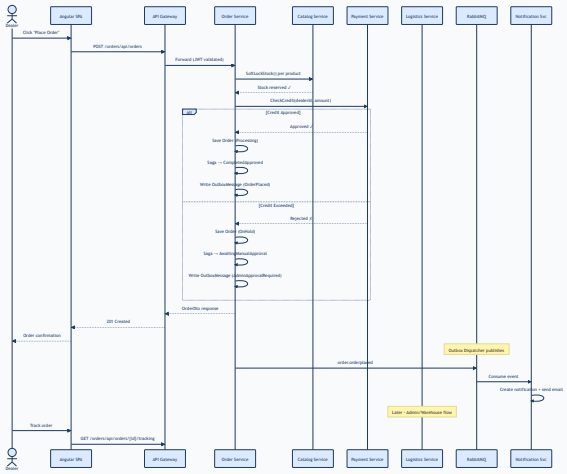


FIGURE 6 — COMPLETE ORDER PLACEMENT FLOW ACROSS ALL SERVICES

11. Event-Driven Architecture & Messaging

11.1 RabbitMQ Topology

COMPONENT	NAME	DETAILS
Exchange	`supplychain.events`	Topic exchange — routing keys like `order.orderplaced`, `logistics.shipmentdelivered`
Queue	`notification.events.queue`	Binds with wildcard `#` — receives ALL events from ALL services
Dead Letter Exchange	DLX	Failed messages (after 5 retries) are moved here for manual inspection
Producers	5 services (all except Notification)	Each has its own `OutboxDispatcher` background service
Consumer	Notification Service only	Single consumer pattern — processes events and fans out to In-App + Email channels

11.2 Event Catalog

SERVICE	ROUTING KEY PATTERN	EVENTS
Identity	`identity.*`	DealerRegistered, DealerApproved, DealerRejected, PasswordResetRequested, PasswordResetCompleted
Catalog	`catalog.*`	ProductCreated, StockUpdated, LowStockAlert
Order	`order.*`	OrderPlaced, AdminApprovalRequired, OrderApproved, OrderCancelled, ReturnRequested, ReturnApproved
Logistics	`logistics.*`	ShipmentCreated, ShipmentAssigned, ShipmentStatusUpdated, ShipmentDelivered
Payment	`payment.*`	InvoiceGenerated, PaymentCaptured, PaymentFailed, CreditLimitUpdated

12. Performance, Scalability & Resilience

⏱️ Performance

Target p95 read latency < 300ms, write latency < 500ms. Redis caching for frequently read data. EF Core compiled queries for hot paths.

📈 Horizontal Scalability

Stateless API services scale via RoundRobin LB. RabbitMQ decouples producers from consumers. Each service independently deployable.

🛡️ Resilience

Circuit breaker (Polly) prevents cascade failures. Outbox pattern survives broker outages. Retry with exponential backoff on transient failures.

🔄 Eventual Consistency

Strong consistency within each service (SQL transactions). Eventual consistency across services via Outbox + RabbitMQ events.

🔒 Idempotency

Redis-backed idempotency keys prevent duplicate order processing from retries or double-clicks. Outbox dispatcher uses status tracking.

📊 Observability

X-Correlation-Id traces requests across all 6 services + gateway. Serilog structured logging. Health check endpoints per service.

13. Project Metrics & Complexity

Backend Metrics

Frontend Metrics

METRIC	VALUE
Microservices	6 domain services + 1 gateway
Databases	6 SQL Server databases
API Endpoints	80+ REST endpoints
EF Core Migrations	20+ migrations total
Domain Events	20+ event types
Background Services	6 outbox dispatchers + email dispatcher
Ocelot Routes	35+ route configurations

METRIC	VALUE
Framework	Angular 21 (standalone)
Feature Modules	10 domains
Components	24 route components + 5 shared
API Services	7 HTTP services
Interceptors	5 functional interceptors
Business Services	12 domain services
Total Source Files	69 TypeScript files (~650 KB)

14. Key Challenges & Solutions

CHALLENGE	PROBLEM	SOLUTION IMPLEMENTED
Distributed Transactions	Order placement touches 3 services (Catalog, Payment, Order) — can't use a single DB transaction	Saga Orchestration pattern with `OrderSagaCoordinator`. Compensating transactions (stock release, credit reversal) on failure. Persisted saga state for auditability.
Event Delivery Reliability	If RabbitMQ is down when an order is placed, the notification is lost	Transactional Outbox pattern — events are persisted atomically with business data. Background dispatcher retries up to 5 times with dead-letter fallback.
Concurrent Stock Updates	Two dealers ordering the last 10 units simultaneously could oversell	Two-phase stock model: `SoftLock` (reserve) → `HardDeduct` (confirm on dispatch). Optimistic concurrency via EF Core row versioning.
Token Refresh Race Condition	Multiple parallel API calls hitting 401 simultaneously, each trying to refresh the token	Frontend `errorInterceptor` uses a `BehaviorSubject` gate: first 401 triggers refresh, all subsequent 401s queue and wait for the new token, then retry.
Cross-Service Email Resolution	Notification service receives events with `userId` but needs email address — owned by Identity service	Internal HTTP call from Notification → Identity API (protected by `X-Internal-Api-Key` header, not JWT).
Role-Adaptive Frontend	5 different roles need different dashboards, menus, and action buttons from the same codebase	Signal-based `AuthStore.role()` drives conditional rendering throughout. Route guards prevent unauthorized access. API services return role-scoped data.
CORS in Development	Angular (:4200) and Gateway (:5000) on different ports — browser blocks cross-origin requests	Angular dev proxy (`proxy.conf.json`) forwards all API paths to :5000 as same-origin. CORS configured only on Gateway for production.

15. Lessons Learned & Interview Talking Points

15.1 Architecture & System Design

Why Microservices over Monolith?

Each domain (Identity, Catalog, Order, Logistics, Payment, Notification) has distinct scaling needs and deployment cadences. Orders need high availability; Notifications can tolerate queuing. Independent deployment reduces blast radius.

Why Saga over 2PC?

6 separate databases make 2PC impossible — it would require XA-compatible transactions across all DBs, blocking locks, and single-point-of-failure coordinator. Saga provides eventual consistency with compensating actions, maintaining high availability.

Why Outbox over Direct Publish?

Direct publish creates dual-write problem: DB write succeeds but RabbitMQ publish fails = lost event. Outbox guarantees atomic event persistence with business data. The dispatcher is the reliability bridge.

Why API Gateway?

Single entry point simplifies frontend (one base URL), centralizes cross-cutting concerns (auth, rate limiting, CORS, circuit breaker), and enables load balancing without client changes.

15.2 Frontend & Angular Specifics

Why Signals over NgRx?

Angular Signals provide reactive state management without the boilerplate of actions, reducers, effects, and selectors. For this app's state complexity (auth + cart + loading), signals with `computed()` are sufficient and more maintainable.

Why Functional Interceptors?

Angular 21 deprecates class-based interceptors. Functional `HttpInterceptorFn` interceptors are simpler, tree-shakable, and align with standalone component architecture — no NgModule registration needed.

How does Silent Token Refresh work?

Error interceptor catches 401 → calls `/identity/api/auth/refresh` with `withCredentials: true` (sends httpOnly cookie) → receives new JWT → updates AuthStore → retries original request. A `BehaviorSubject` ensures only one refresh happens if multiple requests fail simultaneously.

How is the CartStore persisted?

Uses Angular `effect()` to automatically serialize cart items to `localStorage` on every signal change. On initialization, it hydrates from localStorage with full sanitization and validation of each item.

15.3 Backend & .NET Specifics

How does the Order State Machine enforce valid transitions?

The `OrderAggregate` (DDD aggregate root) maintains a `CanTransitionTo()` method with an explicit transition map. Invalid transitions throw `InvalidOperationException`. Status history is tracked in a separate table for audit trail.

How is stock consistency maintained?

Two-phase model: SoftLock on order placement (reserves stock), HardDeduct on dispatch (permanently removes). If soft-lock expires before dispatch, the system re-locks and retries once. On cancellation, locks are released. On return approval, stock is restocked.

How are concurrent modifications handled?

EF Core optimistic concurrency with row version columns. On `DbUpdateConcurrencyException`, the operation returns false rather than throwing — the caller can retry or report conflict.

Summary: This project demonstrates end-to-end design and implementation of a **production-grade distributed system** — from microservice decomposition and event-driven integration to a premium Angular SPA with role-based access — solving real B2B supply chain problems with industry-standard patterns and practices.